Robust Agentic AI for Public Administration: A Multi-Agent Architecture and Controlled Fault Injection Evaluation

Radu-Mihai Gheorghe[1,2] Petru-Liviu Bouruc[2] Ciprian Paduraru[2] Alin Stefanescu[2,3]

[1]UiPath, Romania

[2]University of Bucharest, Romania

[3]Institute for Logic and Data Science, Romania

30th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2026)

Artifact: `github.com/radugheo/agentic-public-administration`

The one-slide summary

Problem

An agentic assistant for public services can return a fluent, confident answer built on a service response that **timed out, arrived incomplete, or came back corrupted**. In a regulated setting, **nothing signals that anything went wrong**.

Idea

Build the assistant as a multi-agent state machine over the real fiscal workflows, then **instrument every external service boundary** with controlled fault injection (timeout, delay, partial response, corruption) and classify what the citizen actually sees.

Result (100 boundary calls, five injection rates)

63% of injected faults pass silently, stable across every non-zero rate and matching the closed form $0.25 + 2 \times 0.25 \times 0.75 = 0.625$. That rate is a property of the **response schema** rather than an implementation defect. Inference costs about \$0.05 per session.

PART 1

Motivation

Fragmented services, one citizen

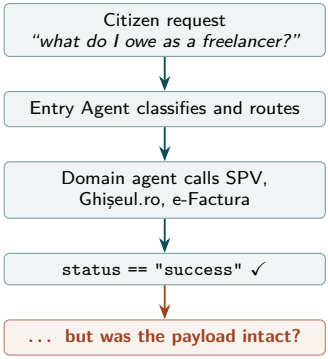
Romania has digitised its fiscal services, but each portal keeps its own interface, vocabulary and procedural logic:

- **SPV**: declarations and fiscal record
- **Ghișeul.ro**: payments
- **RO e-Factura**: national e-invoicing

One freelancer, one year: declare annual income, pay pension and health contributions, register a rental contract, issue an electronic invoice. Four procedures, three portals, no shared state.

Consequence

The arithmetic is trivial. What costs the citizen time is **coordinating the procedures and interpreting the rules**, and that is the part an agent can absorb.



Two gaps

1. Operational coverage $\neq$ evidence of robustness

Published work on AI for tax and government services stops at advisory question answering and single-agent document processing. No system integrates orchestration, domain routing, retrieval, deterministic computation and payment for a concrete national setting, and none reports what happens when those services misbehave.

2. Boundary faults are invisible to agent control flow

Fault injection has reached LLM agents, but in generic or synthetic testbeds. No study applies it to a regulated public service, measures the fraction of faults that propagate silently, or ties that fraction to the structure of the response schema.

Both gaps concern the boundary between a probabilistic model and a deterministic service contract.

A concrete request

The task (small)

"I earned 150,000 lei as a freelancer last year. What do I owe?"

- classify the intent and route it
- compute the pension contribution (25%) and the health contribution (10%) against the 2026 thresholds
- ground the explanation in the Romanian Tax Code
- submit the D212 declaration through SPV and confirm

What the citizen never sees

Every one of those steps crosses a service boundary. The SPV call can

- time out
- return several hundred milliseconds late
- come back with one field set to `None`
- come back with one field replaced by a sentinel

and in all but the first case the agent **usually answers as if nothing happened**.

The paper measures how often that happens, and what governs the number.

Contributions

1. **Architecture.** A multi-agent system for public administration: an Entry Agent classifies intent and routes to one of five domain agents over four shared services.
2. **Instantiation.** Romanian tax and fiscal administration on the UiPath Python SDK and LangGraph, covering D212 filing, property sale, rental income, fiscal certificates and RO e-Factura.
3. **Fault injection framework.** 19 external service boundary operations instrumented with four chaos operators, at configurable rates, replayable from a seed.
4. **Result.** A structural silent pass-through rate of **63%**, stable across injection rates and explained in closed form by the operator mix and the response schema.
5. **Artifact.** Open implementation and evaluation setup.

`https://github.com/radugheo/agentic-public-administration`

PART 2

The system

Five layers, one state machine

Orchestration: the Entry Agent

Sole entry point. Structured LLM classification into intent, confidence, reasoning and extracted entities. A fixed table maps intent to a graph node, and low confidence diverts to clarification.

Shared services: four capabilities

Document intake (OCR and schema validation), RAG over the Romanian Tax Code, deterministic calculation in decimal arithmetic, payment through Ghișeul.ro.

Domain agents: five workflows

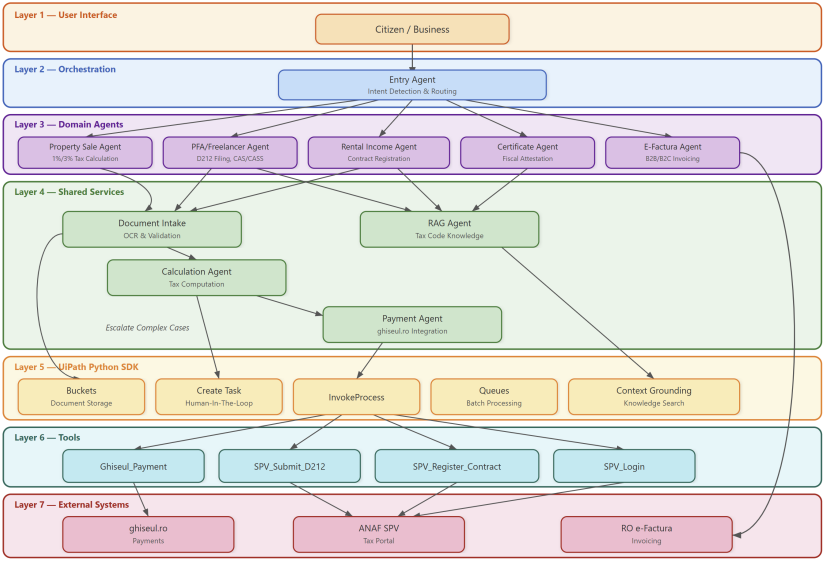
PFA/freelancer (D212), property sale, rental income, fiscal certificate, RO e-Factura. Each takes the shared state, calls shared services, returns updated state.

Infrastructure: UiPath Python SDK

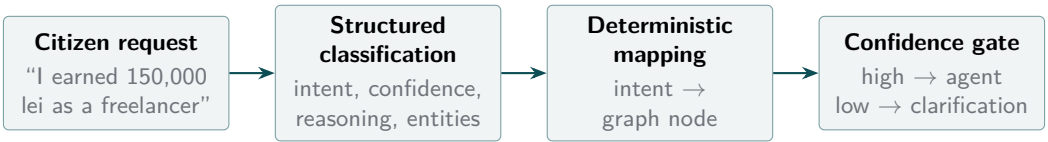
Model gateway, retrieval indices, storage buckets, queues, human review tasks. Government boundaries are mocked, keeping the request and response schemas of the real services.

Hierarchical supervisor pattern, implemented as a LangGraph `StateGraph` with typed shared state and conditional routing.

The architecture



Orchestration: one entry, confidence-gated routing



- The LLM classifies; it does not decide the route. A fixed table turns the predicted intent into a graph node, so the routing step itself stays inspectable.
- Below the confidence threshold the request goes to clarification, even when an intent was assigned. Dispatching the wrong specialist here means **computing the wrong tax**.

The fault injector

Mechanism

A decorator wraps each mocked boundary. **One Bernoulli draw per call** decides whether a fault fires; if it does, exactly one operator is applied to that call. Every event is logged with the operation and the operator type.

Parameters

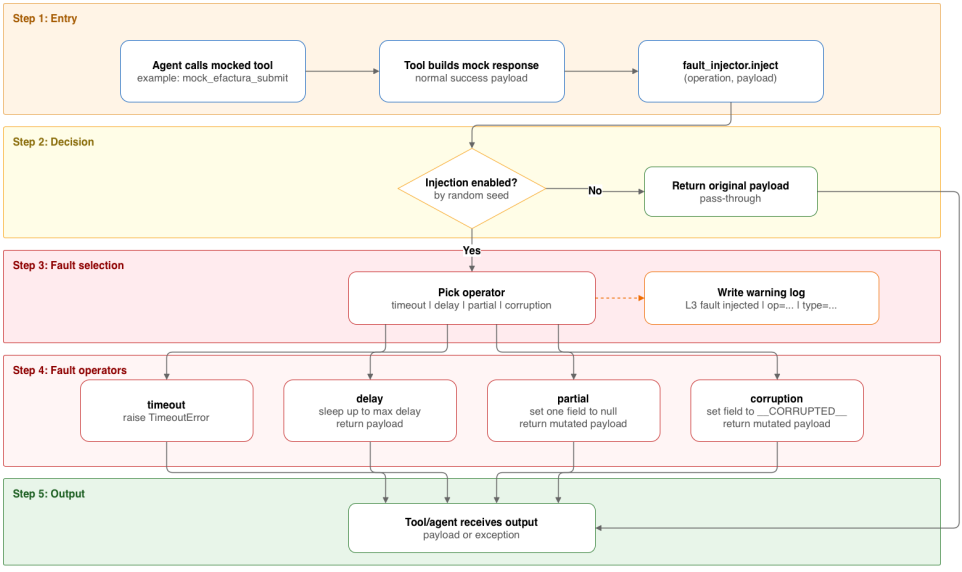
s: seed, for deterministic replay
η: per-call injection rate
$\Delta t_{\max}$: bound on the induced delay

At $\eta = 0$ the nominal system is recovered unchanged.

Operator	Effect	Visible?
Timeout	raises `TimeoutError`	**always**
Delay	sleeps, payload intact	**never**
Partial	one field $\rightarrow$ `None`	**usually not**
Corruption	one field $\rightarrow$ sentinel	**usually not**

19 instrumented boundary operations: 8 function-level mock tools and 11 class-method wrappers across three mocked external systems. Operators are drawn uniformly, so each accounts for a quarter of injected faults. The injector is a separate wrapper layer; the nominal service logic is untouched.

Injection decision flow



Why the faults are silent

Agent control flow usually turns on a single field: a predicate such as `status == "success"`. The rest of the response object carries content the branch never inspects.

`status`	`message`	`id`	`timestamp`
branching field	content only	content only	content only

A single-field perturbation that misses `status` leaves the predicate true, so the agent takes the success branch anyway. For a response with $|\mathcal{F}|$ fields, one of which gates control flow:

$$P(\text{silent}) = 1 - \frac{1}{|\mathcal{F}|}$$

Expected silent rate

$$\underbrace{0.25}_{\text{delay}} + \underbrace{2 \times 0.25 \times 0.75}_{\text{partial, corruption}} = \mathbf{0.625}$$

Timeout, the remaining quarter, always raises.

PART 3

Evaluation

Experimental setup

Prototype

Python 3.11, LangGraph 0.2, GPT-4o through the UiPath LLM Gateway. Every government call goes through the mocked boundary layer, so no live network traffic is required.

Runs

$N = 100$ boundary calls, each an independent trial with at most one injected fault, sampled across the 8 function-level mock tools by call frequency in the five demo workflows.

$\eta \in \{0,\ 0.25,\ 0.50,\ 0.75,\ 1.00\}, \quad s = 17,$
$\Delta t_{\max} = 250\,\text{ms}.$

Detectable

The response signals failure: an exception propagates, or an explicit error or degraded-service notice reaches the user.

Silent pass-through

Anything else. The citizen-facing answer looks normal.

The unit of analysis is the call, not the session. A full workflow crosses several boundaries, each with its own draw.

One trace, in full

A citizen reports PFA income of 150,000 RON and asks what is owed.

1. Entry Agent classifies `pfa_d212_filing` with confidence 0.95
2. PFA Agent computes pension 9,900 RON and health 1,980 RON, total 11,880 RON
3. On the confirmation turn the SPV submission call is intercepted at $\eta = 1.0$, $s = 17$

```
[WARN] fault injected | op=mock_spv_submit_d212 | type=partial
```

The `message` field of `SPVSubmissionResult` was set to `None`. `status` stayed `"success"`, so the agent followed the success branch and answered:

What the citizen saw

"The D212 declaration was successfully submitted! ID: D212-01B66C85C3, Timestamp: 2026-03-27T22:20:32."

Four fields, one of them gating: a single-field fault stays invisible with probability 0.75. The same arithmetic applies to any response whose branch reads one field.

Fault injection results

η	Faults	TO	DL	PT	CR	Silent	Detectable
0.00	0	0	0	0	0	0 (n/a)	0 (n/a)
0.25	25	6	6	7	6	16 (64%)	9 (36%)
0.50	50	12	13	12	13	32 (64%)	18 (36%)
0.75	75	19	19	18	19	47 (63%)	28 (37%)
1.00	100	25	25	25	25	**63 (63%)**	37 (37%)

$N = 100$ calls, $s = 17$, $\Delta t_{\max} = 250$ ms. TO, DL, PT, CR: timeout, delay, partial, corruption.

- The measured rate sits on the predicted **0.625**. Raising η raises the *count* of silent faults, but the proportion does not move.
- Delay faults added 125.5 ± 72.2 ms per affected call, matching the mean of $\mathcal{U}[1, 250]$; at $\eta = 0.50$ roughly 65 ms per scenario.
- Because the rate follows from the operator mix and the schema, it can be **predicted before the system runs**, and lowered by changing what the branch inspects.

Cost feasibility

Per session
$0.05
one classification, two to three domain turns, one retrieval step

Small office
$51
per month, 50 sessions a day

Large agency
$5,060
per month, 5,000 sessions a day

- About 20,000 input and 2,100 output tokens per session, priced at $1.25 per million input and $10.00 per million output tokens, over 22 working days a month. Licensing, infrastructure and maintenance are excluded.
- Cost scales linearly and stays modest. Routing with a smaller model, keeping the capable one for domain reasoning, pushes it lower.

What stands in the way of deployment is assurance, not arithmetic.

Analysis and limitations

What the rate does and does not depend on

It is fixed by the operator mix and the number of fields in the response object, so it holds at every injection rate. It says nothing about how often faults occur in production, only about what fraction of them the agent can see.

Where the fix belongs

Not in the branch, which is already correct with respect to the flag it reads, but in an **output contract** evaluated after the response is assembled and before it reaches the citizen.

Limitations

- Government boundaries are schema-faithful mocks, not live portals; latency and failure distributions in production will differ.
- The unit of analysis is a single boundary call. Faults compounding across a full citizen session are not measured here.
- Detectability is judged from the citizen-facing response, a behavioural proxy rather than a formal correctness criterion.

PART 4

Conclusion

Conclusion

What the evaluation shows

- A silent pass-through rate of **63%**, stable across injection rates and matching a closed-form prediction.
- The cause is structural: response schemas and branching predicates, not incidental bugs. **Interface coherence is not evidence of execution correctness.**
- Inference cost stays modest at institutional volumes.

Contribution

An operational multi-agent architecture for a national fiscal setting, together with a boundary-level fault study that explains *why* faults pass silently rather than only reporting that they do.

Future work

Output contracts checked before a response reaches the citizen; trace-based assurance with structured message-action traces, deterministic replay and action-level governance; extension to other public-service domains over the same orchestration and shared services layers.

Thank you

Questions?

Artifact: implementation and evaluation setup

https://github.com/radugheo/agentic-public-administration

Radu-Mihai Gheorghe · Petru-Liviu Bouruc · Ciprian Paduraru · Alin Stefanescu

UiPath · University of Bucharest · Institute for Logic and Data Science

Partially supported by the "Romanian Hub for Artificial Intelligence (HRIA)" project, Smart Growth, Digitization and Financial Instruments Programme 2021–2027, MySMIS No. 351416.

BACKUP SLIDES

Additional material

Backup: the five domain agents

PFA / Freelancer

Declarația Unică (D212). Pension contribution (CAS) 25%, due above 12 minimum gross salaries (48,600 RON in 2026); health contribution (CASS) 10%, due above 6. Submission through SPV.

Property sale

3% of the sale value under three years of ownership, 1% at three years or more. Payment guidance through Ghișeul.ro.

Rental income

Contract registration with ANAF and a 10% flat tax on annual rent.

Certificate

Certificat de atestare fiscală: identifies the certificate type, checks document requirements, guides submission.

RO e-Factura

B2B and B2C invoicing: invoice data collection, XML generation to the national standard, submission and status monitoring.

All five produce a standardised response field from the final model output, which keeps formatting consistent across workflows.

Backup: the four shared services

Document intake

OCR plus validation against document-specific schemas for D212 forms, rental contracts and invoices. Returns structured data with confidence scores; below 80% the document is flagged for manual review.

RAG

Retrieval over dedicated indices of the Romanian Tax Code and procedural guidance, so an answer can be traced to a source instead of resting on parametric model knowledge.

Calculation

Decimal arithmetic, no floating point. Rates and thresholds (minimum salary, contribution percentages) are read from configuration, so annual regulatory updates need no code change.

Payment

Builds the payment request from the shared context, initiates it through Ghișeul.ro and returns the transaction details, including the redirect URL.

Design rule: the model interprets, deterministic code computes.

Backup: where the work sits

Agentic orchestration

LangGraph models control flow as a typed state machine: explicit and auditable, unlike open-ended conversation. That matters when the service is regulated.

Retrieval

Tax law is specific and amended often. Retrieval grounds explanations in the Romanian Tax Code and stays separate from the deterministic computation.

Chaos engineering

Deliberate faults, observed effects. Recently applied to LLM agents, but usually in generic or synthetic testbeds.

Closest neighbours

Work on faults in agentic repositories identifies the boundary between probabilistic model output and deterministic program logic as a dominant failure class. That is the same interface targeted here. Reliability benchmarks for LLM agents include timeout, partial-response and schema-drift conditions, but do not connect the observed silent rate to schema structure in a deployed public-service setting.

Backup: cost model

Institution size	Sessions/day	Sessions/month	Cost/month
Small office	50	1,100	$51
Medium office	500	11,000	$506
Large agency	5,000	110,000	$5,060

22 working days per month; $1.25 per 1M input and $10.00 per 1M output tokens.

- Per session: one Entry Agent call (≈ 2,000 input / 200 output tokens), two to three domain turns (≈ 5,000 / 500 each), one retrieval-grounded step (≈ 3,000 / 400). About 20,000 input and 2,100 output tokens in total.
- Costs scale at roughly $1.01 per daily session per month.
- The prototype runs GPT-4o; the projection uses the pricing of a stronger model as an illustrative deployment scenario.

Backup: reproducibility

Deterministic replay

The injector draws from a seeded generator, so a fixed $(s, \eta, \Delta t_{\max})$ reproduces the same sequence of operators over the same sequence of boundary calls. Runs reported here use $s = 17$.

Fault log

Each injected event is written as a warning naming the operation and the operator, which is what makes the qualitative trace on the earlier slide checkable:

```
[WARN] fault injected | op=...  | type=...
```

Turning injection off

Setting $\eta = 0$ returns every payload unchanged, so the same code path serves both the nominal system and the perturbed one. The injector never modifies the mock service logic itself.

Implementation and evaluation artefacts:

`github.com/radugheo/agentic-public-administration`